

Captable Multitenancy: Status and Plan (v2)

Date: 2026-10-05 · **Branch:** `feat/multitenancy-phase0` (22 commits ahead of `main`, not pushed) · **Tests:** 255 passing · **Production build:** passes

Replaces: the v1 plan (same path, see git history). v1 mixed a plan with a log and its code samples went stale; this version is status plus remaining work only.

Goal: make tenant isolation structural, enforce roles inside a tenant, and link each company to the TIN Hub's tenant identity without duplicating tenant logic.

Stack: Next.js 14, tRPC 10, Prisma 5.14, NextAuth 4 (JWT), Hono REST, Postgres (Neon), Vitest 1, Biome.

1. Principles (learned the hard way)

1. **Inventory the perimeter, not just the routers.** The scoped client protects code that uses it. Most real holes were elsewhere: ids supplied as data, unauthenticated server actions, an empty-session login bypass, pages that query directly. Every entry point is listed in section 3.
2. **A guard that is not run is not a guard.** No CI workflow currently runs tests or lint (`.github/workflows/*` only builds an image). Section 6 fixes that.
3. **Tests first, reviewed by a second reader.** Each batch was written test-first and then independently reviewed; the reviews found the three most serious issues.
4. **Ownership split (prevents duplicated tenant logic):**

Concern	Owner
Who a person is, SSO, platform operators	Hub
Which tenant a customer is	Hub (Captable stores <code>hubTenantId</code> later)
Membership, role, status in a company	Captable (<code>Member</code>)
What a role may do on cap-table data	Captable (<code>CustomRole</code> , RBAC)
Tenant-scoped data access	Captable (<code>tenantDb</code>)

Operating rules adopted from the platform documents

Source: the Shared Hubs Baseline execution spec, ADR-0002 and the decision register (see section 11).

5. **Table ownership (ADR-0002).** A table has exactly one owning package. Captable never creates or alters a table owned by a Hub package (`users` , `tenants` , `roles` , `workspaces` , `audit_logs` , `partners` , `role_bindings` , `impersonation_grants`). The link to a Hub tenant is a plain reference on a Captable-owned table (`Company.hubTenantId`). Migrations stay additive where a shared database is involved.
6. **A recorded decision for the fork (decision register D7).** Keeping Captable's own tenancy is a fork of a capability the Hub provides; it is recorded in `docs/adr/0001-captable-keeps-its-own-tenancy.md` with a reason, an owner (to be named) and a reconciliation trigger.
7. **Human gates.** The run stops and waits at each gate, writing the request first:

Gate	Before	Evidence to present
G0	Pushing a branch or opening a PR	commit list, test totals, build result
G4	Applying any migration to a remote database (Neon, production)	backfill report output, rollback note
G5	Publishing, deploying, or enabling a Hub link	smoke-test result, decision record approved

8. **Evidence and retrospective.** After each phase: append one line per task to an evidence log (task, commit, test totals, reviewer verdict) and write a short retrospective (completed, changes needed to the plan, decision: continue, adapt or pause).
9. **Cross-family review on Tier 1 tasks.** Roles, tokens and secrets are reviewed by a reader from a different model family than the author (the Codex-based review skill), in addition to the same-family independent review used so far.
10. **Working style (Karpathy guidelines, as the KB describes them).** State assumptions, keep each change surgical, and define "done when" as checks that are actually run. The skill itself is not installed in this environment; the three principles are applied by name.

2. Architecture today

Graph of `src/` (graphify, rebuilt today): 534 files · 2,196 nodes · 7,358 edges · 90 communities.

`withTenant` is now a hub node; `checkMembership` no longer is.

Layer	What it does	Where
<code>tenantDb(db, companyId)</code>	Prisma client extension: forces <code>where.companyId</code> , stamps creates, scopes child models through their parent, rejects raw SQL	<code>src/server/tenant-db.ts</code>
<code>withTenant</code>	tRPC base procedure: session, then one ACTIVE-membership lookup, then <code>ctx.tenant {db, companyId, memberId, role, customRoleId}</code>	<code>src/trpc/api/trpc.ts</code>
<code>withAccessControl</code>	role check on top of <code>withTenant</code> (one membership lookup per request)	same
Guards	<code>assertTenantOwns</code> (ids supplied as data), <code>assertBucketUsable</code> (ownership)	<code>src/server/tenant-guard.ts</code>
REST	each auth middleware sets <code>tenantDb</code> from the verified member row	<code>src/server/api/middlewares/*</code>
Uploads	authenticated presign and download procedures; <code>Bucket.companyId</code>	<code>src/trpc/routers/bucket-router</code>
Tests	harness (local DB only), 75 tRPC isolation cases, 26 REST cases, upload tests, architecture guard	<code>tests/tenant/*</code>

Router coverage: 37 files on `withTenant`, 19 on `withAccessControl`, 9 `withAuth` and 8 `withoutAuth` (tenantless or token flows by design).

3. Entry-point inventory

Entry point	Protected by	Status
tRPC routers	<code>withTenant</code> / <code>withAccessControl</code> ; architecture rules (a), (b), (c), (e)	done
REST <code>/v1/{companyId}/...</code>	scoped client in middleware; rule (d); REST tests	done
REST cookie auth	rejects <code>{}</code> and partial sessions	done
Server actions / "use server"	file-uploads no longer a server module	done; page-level directives remain (hygiene)

Entry point	Protected by	Status
Server components (4 dashboard pages)	JWT company + global db ; layout re-checks membership	open (S4)
Public token flows (e-sign, data room, updates)	signed token binds the resource	safe by design; tokens never expire
Queue jobs / esign services	trusted server payloads, global db	safe by design
S3 objects	Bucket.companyId , authenticated presign and getUrl	done; legacy NULL-owner buckets unusable

4. Shipped

Commit	What
c421cf8 (on main , PR #1)	accept-invite takeover, cross-tenant writes, REST company leak, ACTIVE-member checks
6a07a81 , adc0f0a (PR #4, open)	e-sign requires the signed token; Google linking, onboarding email overwrite, NEXTAUTH_SECRET fallback
d3fb476 , d2c9c49	test harness with a local-DB guard; cross-tenant matrix
004ef28 , 21b49f1	tenantDb , withTenant ; architecture guard
25ee605	14 client-supplied-id holes (buckets, recipients, roles, switchCompany)
3790663 , 19bfa41 , aa7b726	all routers moved to the tenant client; revoke-invite fixed
dcf9e3a	REST scoped; three REST holes; route collision (/shares/{id})
19548a4	empty session authenticated as an arbitrary member (REST cookie path)
556c42e	update editor page cross-tenant read; three missed routers
6889d6c	unauthenticated file upload/download/delete; bucket owner and backfill

Test count by step: 13, 88, 115, 127, 131, 155, 159, 178, 193.

5. Open findings (ranked)

#	Finding	Severity	Phase
F1	Role checks: mutations done (07724f3 , b1a1cbb). Remaining: REST write routes still allow any active member holding an API token; buttons are not hidden in the UI; existing CUSTOM roles lack the five new subjects (operator remedy needed)	Medium	2
F2	4 dashboard pages read tenant data from the JWT company with the global client (stale after deactivation; layout and page render concurrently)	Medium	2
F3	Public-flow tokens (e-sign, data room, update links) never expire	Medium	2
F4	Member.role default removed; onboarding and seeds set the role explicitly (6197ab6). Done	Done	2
F5	getCompanyList and REST company/getMany list memberships of any status	Low	2
F6	Presigned PUTs have no size limit; Bucket has no DB-level relation to Company	Low	3
F7	Stakeholder.email is globally unique (one person cannot be a stakeholder of two companies; leaks existence)	Medium	3

#	Finding	Severity	Phase
F8	REST: cookie auth fails for requests with a body; header schema demands <code>Authorization</code> for cookie callers; pagination <code>limit</code> default ignored	Low	3
F9	"use server" left on 5 page files (not a hole; hygiene)	Low	3
F10	Keeping Captable's own tenancy is a fork of a Hub capability with no recorded decision (register D7)	Medium	2

6. Remaining plan

Scope rule (ponytail): each task is the smallest change with a test that fails first. Tasks marked **D** need a decision from you (section 7).

Phase 2: roles and perimeter (next)

- [x] **T1 (F4, check first), done 6197ab6**. Confirm every `member.create/upsert` passes a role explicitly; then remove the `@default(ADMIN)` (migration, onboarding passes `ADMIN` for the first member). Test: a member created without a role has no permissions.
- [x] **T2 (F1, D2), done 07724f3 ; ADMIN-grant guard b1a1cbb**. (Original scope:) Add RBAC subjects `securities`, `cap-table-settings`, `updates`, `data-rooms`, `templates`; put `.meta({policies})` on every mutation and on `bucket.getUrl/presignUpload/create (documents)`. Refuse removing, deactivating or demoting the last ADMIN. Test: a CUSTOM role with no grants is denied each mutation (extend the matrix with a "same tenant, insufficient role" column); ADMIN allowed; last-admin refused.
- [] **T3 (F2)**. `getServerTenant()` (cached; `{companyId, db: tenantDb}` from `getPermissions`); use it in the 4 dashboard pages. Add a rule to the architecture test: pages under `src/app/(authenticated)` must not import the global `db`.
- [] **T4 (F3, D)**. Add `exp` to public-flow tokens (e-sign, data room, update link); expired token returns 401. Needs a decision on lifetimes (suggest 30 days, renewable by resend). **Also verify tokens against the current and the previous secret** (as the Hub does for its scope secret), so `NEXTAUTH_SECRET` can be rotated without invalidating live e-sign, data-room and update links; document the rotation steps.
- [] **T5 (F5)**. Filter `getCompanyList` and REST `getMany` by ACTIVE, onboarded members.
- [] **T6a (F10)**. Review and approve `docs/adr/0001-captable-keeps-its-own-tenancy.md`: name the owner, decide the audit question; then (optionally) publish it to the knowledge base. Decision record only, no code.

Phase 3: hygiene

- [] **T6 (F7)**. `Stakeholder` unique on `(companyId, email)`; update callers using the old unique key.
- [] **T7 (F6)**. Add `Bucket -> Company` relation; presign size limit (`Content-Length` condition).
- [] **T8 (F8, F9)**. Fix REST cookie auth with a body, header schema, pagination default; drop the page-level "use server" directives.

Phase 4: Hub linkage (gated, not scheduled)

Opens only when the Hubs provide: a published signing key (JWKS), a claims contract with `tenant_id`, and a membership read endpoint or webhook (none exist today), plus decision D3. Then: an OIDC or WorkOS provider mapping IdP organisation to `Company.hubTenantId`; a signed membership webhook; operator access through a Hub claim only. **Decision:** the `hubTenantId` / `hubUserId` columns (old Task 13) are dropped from this plan until then; unused columns are speculative.

Also at the gate: platform-operator access to a tenant uses the Hub's audited, reason-bearing, time-limited grants (a Hub claim), never a role name in Captable's database; decide whether Captable's audit events are

shipped to the Hub's `audit_logs`; list Captable in the cross-hub register as a non-consumer with the recorded divergence.

Phase 5: row-level security (deferred)

Trigger: a second isolation bug found despite `tenantDb`, or a compliance need. Approach is a documented Prisma pattern: a client extension that runs `set_config('app.current_company_id', ...)` in a transaction with each query, plus `CREATE POLICY ... USING (companyId = current_setting(...))` on the 18 tenant tables.

Rollout and operations (before and after merge)

1. **Gate G0. Merge PR #4**, then rebase this branch onto `main` and open one PR (about 80 files; split by phase if reviewers prefer). **Every PR targets `main`** (the stacked PRs landed in the wrong branch once).
2. **Database (gate G4)**: the `Bucket.companyId` and `Member.role` migrations are applied only to the local test DB. Before any database that holds files: run `docs/bucket-owner-backfill-report.sql` (read-only) and review the buckets that would stay unowned. Neon is empty, so applying there is trivial. Production migration is run by a person with credentials, never delegated.
3. **Config**: `NEXTAUTH_SECRET` must be 32+ characters; the app now refuses to start otherwise.
4. **Behaviour changes to announce**: non-members get `UNAUTHORIZED` instead of `{success:false}`; REST share update is now `PATCH /v1/{companyId}/shares/{id}`; Google sign-in no longer merges into an existing password account.
5. **CI (new)**: add a workflow job that starts a Postgres service, runs `prisma migrate deploy` against it, then `pnpm test`, `tsc --noEmit` and `biome check`. Without it the isolation tests protect nobody.
6. **Evidence and retrospective (per phase)**: append the evidence lines and write the retrospective described in section 1.
7. **Smoke test (manual, once per release)**: upload and open a document; sign a document through an emailed link; invite and accept a member; switch company; open a data room link. These flows are covered by tests and the build, never by a browser run.

7. Decisions needed

#	Decision	Default in this plan	Blocks
D1	Stay on NextAuth and Prisma; link to the Hub by tenant id	Stay	Phase 4
D2	Role matrix for CUSTOM roles on cap-table subjects	Read for all members; writes need a grant; member management admin-only	T2
D3	Which login provider the Hub fronts (Supabase Auth or WorkOS)	Unknown	Phase 4
D4	Public-link token lifetime	30 days	T4
D5	One large PR or one per phase	One per phase	Rollout
D6	Named owner for the tenancy fork (register D7) and approval of ADR-0001	Unknown	T6a, Phase 4
D7	Ship Captable audit events to the Hub's <code>audit_logs</code> , or keep separate	Keep separate until the Hub has an ingest endpoint	Phase 4

8. Verified against current documentation (Context7)

Claim the plan relies on	Source says	Effect on plan
Prisma treats <code>undefined</code> in a filter as "do nothing" (key omitted)	Prisma docs, null vs undefined	Root cause of the empty-session bypass; explains why <code>checkMembership</code> now rejects falsy ids
Query extensions via <code>\$allModels.\$allOperations</code> can rewrite <code>args</code> ; raw queries reach <code>\$allOperations</code> with <code>model</code> undefined	Prisma client extensions docs	<code>tenantDb</code> design; raw SQL is rejected explicitly
RLS with Prisma: <code>set_config(...)</code> per query inside a transaction, plus policies, is a documented pattern	Prisma client extensions blog	Phase 5 approach
"use server" exports are public endpoints; authenticate and authorize inside each action	Next.js 14 Server Actions and authentication docs	The upload fix (no unauthenticated server actions)
<code>/api/auth/session</code> returns an empty object when there is no session; <code>getSession</code> returns <code>null</code>	NextAuth.js (v4) docs	The cookie path must treat <code>{}</code> as unauthenticated
tRPC middleware extends context with <code>next({ ctx })</code> ; <code>createCaller</code> is for server-side calls and tests, not for calling procedures from other procedures	tRPC server docs	<code>withTenant</code> and the test harness

The Context7 results include Prisma v7 pages while this repo is on 5.14; behaviours above were also confirmed by the test suite on the installed versions.

9. Model routing (remaining work)

Task	Model	Why
T1, T2, T4	<code>opus</code> (+ cross-family review)	Auth, roles, tokens: security paths
T3	<code>sonnet</code> + <code>opus</code> review	Mechanical with a failing rule first
T5, T6, T7, T8	<code>sonnet</code>	Mechanical with a failing test; <code>opus</code> review per phase
T6a	<code>you</code>	Decision record: owner and audit question
CI job, smoke-test checklist	<code>sonnet</code>	Config
Migrations on shared databases, deploys	<code>orchestrator</code>	Credentials; never delegated

Each phase ends with an independent `opus` review of the diff (the reviews found the empty-session bypass, the open editor page and the weak backfill) and a written summary of the previous phase before the next starts.

10. Hub dependencies (not Captable work)

Repo	Needed before Phase 4
<code>shared-identity-hub</code>	Close the <code>x-tenant-id</code> header bypass in <code>get_current_tenant_id()</code> ; one definition of "operator" (four exist); tenant switching and the Next adapter (specified, not built)
<code>tin-boss-api</code>	Tenant and membership read endpoint or webhook; JWKS endpoint; claims revocation

Repo	Needed before Phase 4
shared-client-care-hub	Composite <code>(tenant_id, id)</code> keys on CRM tables; one tenant-resolution function (not needed by Captable)

11. Alignment with the platform documents

Read on 2026-10-05 from the TIN knowledge base (programming.docs.tin.info). Treated as inputs, not instructions.

Document	What this plan adopts
Execution spec, Shared Hubs Baseline (2026-09-22)	Human gates (G0, G4, G5), evidence log and per-phase retrospective, cross-family validation on Tier 1 tasks, two-secret rotation for signed tokens, audited operator grants. Confirms the Hub packages are published and installable, but their isolation still depends on Supabase <code>auth.uid()</code>
ADR-0002, schema table ownership	One owner per table; Captable never alters Hub-owned tables; the Hub link is a reference on a Captable-owned table
Shared platform architecture: decision register	D7: forking a capability needs a recorded decision with an owner and reconciliation plan (ADR-0001); D9: list Captable in the cross-hub register; R1 and R2 are the risks the ADR bounds
Prompt best practice (per-stage plan review)	Method order used here: graphify, Karpathy guidelines, ponytail, then verification before claiming done